

Developer Workflow Cookbook: Conda + uv

Updated January 08, 2026 (America/Chicago)

This cookbook describes a repeatable workflow for using: **Conda** for the base environment and system libraries (Python, OpenSSL, SQLite, CA certificates, etc.) **uv** for Python dependency management (pyproject.toml + uv.lock) while installing packages into the active Conda environment. The goal is: **Conda creates the environment, uv manages and locks Python dependencies**, and you keep Conda in sync by applying the locked requirements with **uv pip sync**.

Key files in the repo

environment.yaml - creates the Conda environment (base + tooling + system deps) **pyproject.toml** - declares Python dependencies (what you want) **uv.lock** - locks resolved Python dependencies (what you actually get) **requirements.locked.txt** - exported pinned requirements used to sync the active Conda env

1. Template: environment.yaml

Use this as a starting point. Prefer conda-forge for consistent builds. Keep Conda dependencies focused on Python itself, system libraries, and common developer tooling. Let uv handle most Python packages.

```
name: YOUR_ENV_NAME
channels:
  - conda-forge
  - defaults
dependencies:
  # Core runtime
  - python>=3.13
  - pip
  - setuptools
  - wheel

  # System / shared libraries (often needed by wheels and SSL)
  - openssl
  - ca-certificates
  - sqlite

  # Developer tooling (CLI)
  - uv
  - git
  - curl

  # Jupyter + kernel support
  - jupyterlab
  - ipykernel
  - ipython

  # Optional: if you have a small number of pip-only bootstrap deps
  # - pip:
  #   - some-pip-only-package
```

Notes

If your org standardizes on **mamba**, the same YAML file works. If you need platform-specific system packages, prefer keeping those in Conda rather than pip/uv. Avoid mixing a large pip section in environment.yaml if uv is your dependency manager. Use uv for Python deps instead.

2. Cookbook: Initial setup from scratch (Conda + uv)

Follow these steps when creating the project environment for the first time on a new machine.

Step 0 - Prerequisites

Install Conda or Mambaforge/Miniforge and ensure **conda** (or **mamba**) is available in your shell. Have Git installed if you plan to clone the repo.

Step 1 - Create the Conda environment from environment.yaml

```
# from the repo root
conda env create -f environment.yaml

# or, if you prefer an explicit name override:
conda env create -f environment.yaml -n YOUR_ENV_NAME

# mamba equivalent:
mamba env create -f environment.yaml
```

Step 2 - Activate and verify

Confirm you are inside the Conda environment before doing anything with uv.

```
conda activate YOUR_ENV_NAME

python -c "import sys; print(sys.executable)"
# should point inside your CONDA_PREFIX
```

Step 3 - Initialize the uv project in the repo (only once per repo)

If the repo already has **pyproject.toml**, skip initialization. Otherwise create it and set a Python constraint.

```
# create a minimal pyproject.toml (if not present)
uv init --bare

# ensure pyproject.toml includes:
# [project]
# requires-python = ">=3.13"
```

Step 4 - Add your initial Python dependencies with uv

Add project dependencies (runtime and dev) using uv. This updates **pyproject.toml** and the lockfile (**uv.lock**).

```
# examples
uv add requests
uv add numpy pandas
uv add --dev ruff pytest
```

Step 5 - Lock dependencies

Lock resolves and pins transitive dependencies into **uv.lock**. Some uv commands lock automatically, but calling it explicitly keeps the workflow obvious.

```
uv lock
```

Step 6 - Export a pinned requirements file for the Conda environment

Export a fully pinned, reproducible set of requirements that pip-style tools can install/sync.

```
# recommended: compile from pyproject into a pinned requirements file
uv pip compile pyproject.toml -o requirements.locked.txt
```

Step 7 - Sync the active Conda environment to match the lock

Apply the pinned set into the currently active Conda environment. This will add and remove Python packages so the environment matches **requirements.locked.txt**.

```
uv pip sync requirements.locked.txt
```

Step 8 - Register the kernel for JupyterLab (optional but common)

This makes the Conda environment selectable as a Jupyter kernel.

```
python -m ipykernel install --user --name YOUR_ENV_NAME --display-name "Python (YOUR_ENV_NAME)"
```

What should be committed to git?

Commit: **environment.yaml**, **pyproject.toml**, **uv.lock**, and (if used for deployments/CI) **requirements.locked.txt**. Do not commit: your local Conda prefix, caches, or platform-specific build artifacts.

3. Cookbook: When you add or change a Python dependency later

Use this flow anytime after initial setup when you need to add, remove, or upgrade a Python dependency. The rule is: **change dependencies via uv**, then **re-export** and **re-sync the active Conda env**.

Step 1 - Activate the Conda environment

```
conda activate YOUR_ENV_NAME
```

Step 2 - Add, remove, or update dependencies using uv

These commands update **pyproject.toml** and then you will re-lock.

```
# add a new runtime dependency
uv add rich
```

```
# add a dev dependency
uv add --dev mypy
```

```
# remove a dependency
uv remove rich
```

Step 3 - Re-lock

Update **uv.lock** to reflect the new dependency graph.

```
uv lock
```

Step 4 - Re-export the pinned requirements file

Regenerate **requirements.locked.txt** from the locked project state.

```
uv pip compile pyproject.toml -o requirements.locked.txt
```

Step 5 - Sync Conda environment to the new lock

Bring the active Conda environment in line with the updated lock file.

```
uv pip sync requirements.locked.txt
```

Step 6 - Sanity check

Run a quick import check or your test suite.

```
python -c "import sys; print(sys.version)"
python -c "import pkgutil; print('ok')"
# or
pytest -q
```

4. Notes on uv sync vs uv pip sync

uv sync is primarily for syncing a uv-managed virtual environment (commonly **.venv**) to match **uv.lock**. When you are intentionally using **Conda** as the environment, you typically keep the Conda env aligned by exporting a pinned requirements file and running **uv pip sync requirements.locked.txt**. uv will detect an activated Conda

environment via `CONDA_PREFIX`, so running `uv pip sync` after `conda activate` will target the Conda env.

5. Troubleshooting quick hits

uv installs into the wrong Python: verify you activated Conda, then re-run `python -c "import sys; print(sys.executable)"`. **Binary wheels fail to build:** prefer Conda packages for compiled libs or add missing system deps to `environment.yaml`. **Need a clean slate:** remove and recreate the Conda env from `environment.yaml`, then run the initial setup steps (lock, compile, sync).